AutoMartX Documentation

AI-Powered Smart Vehicle Marketplace

August 2026

01. Project Title
- Project Name
- Full Project Title
- Project Tagline
- Project Description
- Why the Name “AutoMartX”?
- Vision
- Mission
- Project Objectives
- Key Highlights
- Innovation Statement
- Project Motto

02. Domain
- Primary Domain
- Domain Categories
- Sub-Domains Involved
- Why This Domain Was Chosen
- Domain Summary

03. Who is the User
- Overview
- 1. Buyer
- 2. Seller
- 3. Administrator
- User Comparison Table

04. Problem We Are Solving
- Core Problems Identified
- Summary of the Core Problem
- Who Is Affected
- How AutoMartX Responds to This Problem

05. Proposed Solution
- Solution Summary
- How the Solution Works
- Key Solution Components
- Solution Architecture (High-Level Flow)
- Expected Outcome

06. Core Entities / Database Tables
- Overview
- Entity Relationship Summary
- Table Summary
- Relationships Summary
- SQL Schema (MySQL)
- Database Connection Configuration
- Why This Schema Design

07. User Roles & Permissions
- Role Hierarchy
- Permission Matrix

Authentication & Authorization Approach
Why Role-Based Access Control Matters
08. Success Criteria
Functional Success Criteria
AI Feature Success Criteria
Technical Success Criteria
Measurable Outcomes Summary
09. Out of Scope
Excluded Features (v1.0)
Additional Exclusions
Future Scope Consideration
10. Chosen Track
Selected Track
Technology Stack
Deployment & Hosting
System Architecture Overview
Why This Track Was Chosen
Summary
Appendix: System Architecture
User Workflow
Database Architecture
Hosting & Deployment Architecture

01. Project Title

Project Name

AutoMartX

Full Project Title

AutoMartX – AI-Powered Smart Vehicle Marketplace

Project Tagline

> *“Buy Smart. Sell Secure. Powered by Artificial Intelligence.”*

Project Description

AutoMartX is a next-generation AI-powered vehicle marketplace designed to make buying and selling vehicles safer, smarter, and more transparent. Unlike traditional vehicle marketplace platforms that only display vehicle listings, AutoMartX helps users make informed decisions using Artificial Intelligence. The platform analyzes vehicle information, estimates a fair market price, evaluates the vehicle’s condition, detects suspicious or fake listings, and measures seller credibility before a buyer makes a purchase.

The system combines a modern online marketplace with intelligent decision-support features to improve trust between buyers and sellers.

Why the Name “AutoMartX”?

- **Auto** – Represents automobiles and the automotive industry.
- **Mart** – Represents a digital marketplace where users can buy and sell vehicles.
- **X** – Represents the next generation of innovation, Artificial Intelligence, smart automation, and advanced technology.

Vision

To build a trusted digital marketplace where every vehicle transaction is supported by Artificial Intelligence, transparency, and data-driven decision making.

Mission

- Make vehicle buying safer.
- Help sellers build trust.
- Reduce fraudulent listings.

- Provide AI-powered decision support.
- Create a transparent and user-friendly marketplace.

Project Objectives

- Develop a secure online vehicle marketplace.
- Enable buyers and sellers to connect efficiently.
- Integrate AI-powered price prediction.
- Detect fraudulent vehicle listings.
- Calculate seller trust scores.
- Recommend vehicles based on user preferences.
- Improve the overall vehicle buying experience.

Key Highlights

- Smart Vehicle Marketplace
- AI Fair Price Prediction
- AI Vehicle Health Score
- Fake Listing Detection
- Seller Trust Score
- Nearby Vehicle Discovery
- Wishlist Management
- Vehicle Inspection Booking
- Smart Notifications
- Admin Analytics Dashboard

Innovation Statement

AutoMartX is not just a platform for listing vehicles. It is an intelligent marketplace that combines Artificial Intelligence, trust analysis, and smart recommendations to help users make confident buying and selling decisions.

Project Motto

> *“Don’t just buy a vehicle. Buy it with confidence.”*

02. Domain

Primary Domain

Automotive E-Commerce & Artificial Intelligence

AutoMartX operates at the intersection of the automotive resale industry and modern AI-driven web technology, combining online marketplace functionality with intelligent decision-support systems.

Domain Categories

Automotive / Vehicle Resale — The core business domain: buying and selling used and new vehicles through a digital platform.

E-Commerce & Online Marketplace — AutoMartX functions as a two-sided marketplace connecting buyers and sellers.

Artificial Intelligence & Machine Learning — Used for price prediction, health scoring, fraud detection, and personalized recommendations.

Data Analytics — Vehicle data is analyzed to generate insights such as market trends and pricing accuracy.

Trust & Safety — Seller verification, trust scoring, and fraud detection align the project with trust-and-safety systems.

Full-Stack Web Development — Built as a complete web application covering the full development lifecycle.

Sub-Domains Involved

Sub-Domain	Application in AutoMartX
Web Development	React.js frontend + FastAPI backend
Database Management	MySQL relational schema design
Machine Learning	Price prediction, health scoring, fraud detection models
Cloud Computing	Railway/AWS deployment, Cloudinary storage
Geolocation Services	Google Maps API for nearby vehicle discovery
Authentication & Security	JWT-based secure login system
UI/UX Design	Tailwind CSS-based responsive interface

Why This Domain Was Chosen

- High real-world relevance and demand for trustworthy vehicle marketplaces.
- Strong opportunity to apply Artificial Intelligence in a practical, measurable way.
- Combines multiple technical disciplines — web development, databases, AI/ML, and cloud deployment.
- Addresses a genuine problem (fraud, mispricing, lack of transparency).

Domain Summary

> *AutoMartX belongs to the Automotive E-Commerce domain, enhanced with Artificial Intelligence and Data Analytics, and delivered through a full-stack web application.*

03. Who is the User

Overview

AutoMartX is designed for three primary user groups, each with distinct goals, responsibilities, and permissions within the platform.

1. Buyer

The Buyer is anyone looking to purchase a vehicle through the platform.

What they can do:

- Register and log in securely
- Browse and search vehicles with advanced filters
- Compare multiple vehicles side by side
- Save favourite vehicles to a wishlist
- Contact sellers directly
- Book vehicle inspections
- View AI-generated price predictions
- View AI Vehicle Health Score
- Check Seller Trust Score before engaging
- Receive personalized vehicle recommendations
- Submit reviews after a transaction

Core need: Confidence that the vehicle they buy is fairly priced, in good condition, and sold by a trustworthy seller.

2. Seller

The Seller is anyone listing a vehicle for sale on the platform.

What they can do:

- Create and manage vehicle listings
- Upload multiple vehicle images
- Update vehicle information as needed
- Manage buyer bookings and inspection requests
- Respond to buyer enquiries
- View listing performance analytics
- Receive AI suggestions to improve listing quality
- Verify ownership documents to build trust

Core need: A platform that helps their listings stand out, build buyer trust quickly, and reduces effort in managing enquiries.

3. Administrator

The Administrator maintains the integrity, security, and smooth operation of the marketplace.

What they can do:

- Manage all user accounts (buyers and sellers)
- Verify and approve sellers
- Approve or reject vehicle listings
- Remove fraudulent or duplicate content
- View platform-wide analytics dashboards
- Manage vehicle categories
- Handle reported listings and user complaints
- Monitor overall platform activity
- Generate business and usage reports

Core need: Full visibility and control over the platform to ensure it remains safe and trustworthy.

User Comparison Table

Aspect	Buyer	Seller	Administrator
Primary Goal	Find and purchase a trustworthy vehicle	Sell a vehicle efficiently and credibly	Maintain platform integrity
Key AI Feature Used	Price Prediction, Health Score, Trust Score	Listing Improvement Suggestions	Fraud/Fake Listing Alerts
Access Level	Standard user	Standard user (with listing rights)	Full administrative access
Verification Required	Basic account verification	Ownership document verification	N/A (internal role)

04. Problem We Are Solving

Core Problems Identified

1. Lack of Fair Pricing Transparency — Buyers often have no reliable way to know whether a vehicle's asking price is fair.

2. Fake and Misleading Listings — Many marketplace platforms are filled with duplicate advertisements, manipulated vehicle details, and fraudulent listings.

3. No Visibility into Vehicle Condition — Buyers typically must rely entirely on seller-provided descriptions and photos.

4. No Way to Judge Seller Credibility — Existing platforms rarely provide a reliable, data-driven way to assess whether a seller is trustworthy.

5. Platforms Focus Only on Listing, Not Decision-Making — Most platforms function as simple advertisement boards.

6. Inefficient Search and Discovery — Without smart filtering or personalized recommendations, buyers spend excessive time browsing.

Summary of the Core Problem

> *Existing vehicle marketplaces help users find vehicles — but they do not help users evaluate, trust, or confidently decide on a purchase.*

Who Is Affected

Stakeholder	How They Are Affected
Buyers	Risk overpaying, buying poor-condition vehicles, or falling for fraud
Sellers	Genuine sellers struggle to stand out from fraudulent or low-quality listings
Marketplace Platforms	Loss of user trust and engagement due to unresolved fraud and pricing issues

How AutoMartX Responds to This Problem

Problem	AutoMartX Solution
Unfair/unclear pricing	AI Fair Price Prediction
Fake listings	AI Fake Listing Detection
Unknown vehicle condition	AI Vehicle Health Score
Untrustworthy sellers	Seller Trust Score
Poor decision support	AI-driven insights across the buying journey
Inefficient discovery	Smart Filters + Personalized Recommendations + Nearby Discovery

05. Proposed Solution

Solution Summary

> *AutoMartX combines a modern marketplace with Artificial Intelligence to help buyers evaluate vehicles with confidence and help sellers build trust.*

How the Solution Works

1. **Vehicle Listing** — Sellers list vehicles with detailed information.
2. **AI Analysis Layer** — The system generates a fair price range, health score, fraud risk assessment, and updated trust score.
3. **Informed Buyer Decisions** — Buyers browse AI-enriched listings instead of raw advertisements.
4. **Trust-Building for Sellers** — Sellers receive AI suggestions and document verification.
5. **Safe Transaction Facilitation** — Buyers and sellers connect through messaging, inspections, and finalize deals.

Key Solution Components

- **Smart Vehicle Marketplace** — Listing, browsing, searching, filtering.
- **AI Fair Price Prediction** — Analyzes brand, model, year, mileage, condition, and market trends.
- **AI Vehicle Health Score** — Estimates overall condition from available data.
- **AI Fake Listing Detection** — Flags suspicious listings based on duplicate images, unrealistic pricing, and spam patterns.
- **Seller Trust Score** — Dynamic rating from sales history, reviews, and verification.
- **Personalized Recommendation Engine** — Suggests vehicles based on preferences and history.
- **Nearby Vehicle Discovery** — Google Maps integration.
- **Buyer–Seller Communication & Booking** — Built-in messaging and inspection scheduling.

Solution Architecture (High-Level Flow)

```
Seller Lists Vehicle
        |
        v
AI Analysis Layer (Price, Health, Fraud Risk)
        |
        v
Enriched Listing Published
        |
        v
Buyer Searches / Filters / Views AI Insights
        |
        v
Buyer Books Inspection / Contacts Seller
        |
        v
Transaction Completed -> Review Submitted -> Trust Score Updated
```

Expected Outcome

> *AutoMartX doesn’t just list vehicles — it helps people buy and sell them with confidence.*

06. Core Entities / Database Tables

Overview

The database schema is designed for **MySQL** using **SQLAlchemy ORM** (with the PyMySQL driver).

Entity Relationship Summary

```
Users --+-- Vehicles --+-- Vehicle_Images
        |              +-- AI_Predictions
        |              +-- Bookings
        |              +-- Wishlist
        |              +-- Reviews
        |
        +-- Seller_Verification
        +-- Notifications
        +-- Reports
```

Table Summary

#	Table	Purpose
1	Users	Stores all registered accounts (buyer, seller, admin)
2	Categories	Vehicle categories for filtering
3	Vehicles	Vehicle listings created by sellers
4	Vehicle_Images	Multiple images per listing
5	Bookings	Buyer-scheduled vehicle inspections
6	Wishlist	Vehicles saved by buyers
7	Reviews	Buyer feedback after transactions
8	Seller_Verification	Identity/ownership document verification
9	AI_Predictions	AI-generated price, health, and fraud data
10	Notifications	System-generated alerts
11	Reports	Buyer/seller complaints
12	Payments (Future)	Reserved for future transaction processing

Relationships Summary

Relationship	Type
Users -> Vehicles	One-to-Many
Vehicles -> Vehicle_Images	One-to-Many
Vehicles -> AI_Predictions	One-to-One
Users -> Bookings	One-to-Many (as buyer)
Vehicles -> Bookings	One-to-Many
Users -> Wishlist -> Vehicles	Many-to-Many
Users -> Reviews	One-to-Many (buyer and seller)
Users -> Seller_Verification	One-to-One

Users -> Notifications	One-to-Many
Users -> Reports	One-to-Many

SQL Schema (MySQL)

```sql
-- 1. USERS
CREATE TABLE users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(150) NOT NULL,
    email VARCHAR(150) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    role ENUM('buyer', 'seller', 'admin') NOT NULL,
    phone_number VARCHAR(20),
    profile_image VARCHAR(255),
    is_verified BOOLEAN DEFAULT FALSE,
    trust_score FLOAT DEFAULT 0,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
) ENGINE=InnoDB;

-- 2. CATEGORIES
CREATE TABLE categories (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    description TEXT
) ENGINE=InnoDB;

-- 3. VEHICLES
CREATE TABLE vehicles (
    id INT AUTO_INCREMENT PRIMARY KEY,
    seller_id INT NOT NULL,
    category_id INT,
    brand VARCHAR(100) NOT NULL,
    model VARCHAR(100) NOT NULL,
    manufacture_year INT NOT NULL,
    fuel_type ENUM('petrol', 'diesel', 'electric', 'hybrid'),
    transmission ENUM('manual', 'automatic'),
    kilometers_driven INT NOT NULL,
    `condition` ENUM('excellent', 'good', 'fair', 'poor'),
    price DECIMAL(12, 2) NOT NULL,
    description TEXT,
    location VARCHAR(255),
    status ENUM('pending', 'approved', 'rejected', 'sold') DEFAULT 'pending',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    FOREIGN KEY (seller_id) REFERENCES users(id) ON DELETE CASCADE,
    FOREIGN KEY (category_id) REFERENCES categories(id) ON DELETE SET NULL
) ENGINE=InnoDB;

-- 4. VEHICLE_IMAGES
CREATE TABLE vehicle_images (
    id INT AUTO_INCREMENT PRIMARY KEY,
    vehicle_id INT NOT NULL,
    image_url VARCHAR(255) NOT NULL,
    is_primary BOOLEAN DEFAULT FALSE,
    uploaded_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (vehicle_id) REFERENCES vehicles(id) ON DELETE CASCADE
) ENGINE=InnoDB;

-- 5. BOOKINGS
```

```
        CREATE TABLE bookings (
            id INT AUTO_INCREMENT PRIMARY KEY,
            buyer_id INT NOT NULL,
            vehicle_id INT NOT NULL,
            booking_date TIMESTAMP NOT NULL,
            status ENUM('pending', 'confirmed', 'completed', 'cancelled') DEFAULT
'pending',
            notes TEXT,
            created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
            FOREIGN KEY (buyer_id) REFERENCES users(id) ON DELETE CASCADE,
            FOREIGN KEY (vehicle_id) REFERENCES vehicles(id) ON DELETE CASCADE
        ) ENGINE=InnoDB;

        -- 6. WISHLIST
        CREATE TABLE wishlist (
            id INT AUTO_INCREMENT PRIMARY KEY,
            buyer_id INT NOT NULL,
            vehicle_id INT NOT NULL,
            added_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
            UNIQUE KEY unique_wishlist_entry (buyer_id, vehicle_id),
            FOREIGN KEY (buyer_id) REFERENCES users(id) ON DELETE CASCADE,
            FOREIGN KEY (vehicle_id) REFERENCES vehicles(id) ON DELETE CASCADE
        ) ENGINE=InnoDB;

        -- 7. REVIEWS
        CREATE TABLE reviews (
            id INT AUTO_INCREMENT PRIMARY KEY,
            buyer_id INT NOT NULL,
            seller_id INT NOT NULL,
            vehicle_id INT NOT NULL,
            rating INT NOT NULL CHECK (rating BETWEEN 1 AND 5),
            comment TEXT,
            created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
            FOREIGN KEY (buyer_id) REFERENCES users(id) ON DELETE CASCADE,
            FOREIGN KEY (seller_id) REFERENCES users(id) ON DELETE CASCADE,
            FOREIGN KEY (vehicle_id) REFERENCES vehicles(id) ON DELETE CASCADE
        ) ENGINE=InnoDB;

        -- 8. SELLER_VERIFICATION
        CREATE TABLE seller_verification (
            id INT AUTO_INCREMENT PRIMARY KEY,
            seller_id INT NOT NULL,
            document_type ENUM('id_proof', 'ownership_proof', 'business_license'),
            document_url VARCHAR(255) NOT NULL,
            verification_status ENUM('pending', 'approved', 'rejected') DEFAULT 'pending',
            reviewed_by INT,
            reviewed_at TIMESTAMP NULL,
            FOREIGN KEY (seller_id) REFERENCES users(id) ON DELETE CASCADE,
            FOREIGN KEY (reviewed_by) REFERENCES users(id) ON DELETE SET NULL
        ) ENGINE=InnoDB;

        -- 9. AI_PREDICTIONS
        CREATE TABLE ai_predictions (
            id INT AUTO_INCREMENT PRIMARY KEY,
            vehicle_id INT NOT NULL,
            predicted_price_min DECIMAL(12, 2),
            predicted_price_max DECIMAL(12, 2),
            price_confidence_score FLOAT,
            health_score INT CHECK (health_score BETWEEN 0 AND 100),
            fraud_risk_level ENUM('low', 'medium', 'high'),
            generated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
            FOREIGN KEY (vehicle_id) REFERENCES vehicles(id) ON DELETE CASCADE
        ) ENGINE=InnoDB;
```

```sql
-- 10. NOTIFICATIONS
CREATE TABLE notifications (
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    type ENUM('enquiry', 'booking', 'listing_approval', 'price_change', 'system'),
    message TEXT NOT NULL,
    is_read BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
) ENGINE=InnoDB;

-- 11. REPORTS
CREATE TABLE reports (
    id INT AUTO_INCREMENT PRIMARY KEY,
    reported_by INT NOT NULL,
    reported_vehicle_id INT,
    reported_user_id INT,
    reason TEXT NOT NULL,
    status ENUM('open', 'under_review', 'resolved', 'dismissed') DEFAULT 'open',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (reported_by) REFERENCES users(id) ON DELETE CASCADE,
    FOREIGN KEY (reported_vehicle_id) REFERENCES vehicles(id) ON DELETE CASCADE,
    FOREIGN KEY (reported_user_id) REFERENCES users(id) ON DELETE CASCADE
) ENGINE=InnoDB;

-- 12. PAYMENTS (Future Scope)
CREATE TABLE payments (
    id INT AUTO_INCREMENT PRIMARY KEY,
    booking_id INT NOT NULL,
    amount DECIMAL(12, 2) NOT NULL,
    status ENUM('pending', 'completed', 'failed', 'refunded') DEFAULT 'pending',
    payment_method VARCHAR(50),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (booking_id) REFERENCES bookings(id) ON DELETE CASCADE
) ENGINE=InnoDB;
```

Database Connection Configuration

```
SQLALCHEMY_DATABASE_URL = "mysql+pymysql://<username>:<password>@<host>:
<port>/<database_name>"

# Example
SQLALCHEMY_DATABASE_URL =
"mysql+pymysql://root:password@localhost:3306/automartx_db"
```

Why This Schema Design

- Normalized structure avoids data duplication.
- AI_Predictions kept as a separate table for easy regeneration/versioning.
- Role-based Users table simplifies authentication for three user types.
- AUTO_INCREMENT integer IDs keep joins fast on MySQL's InnoDB engine.
- ON DELETE CASCADE keeps related records consistent.
- ENUM types enforce data integrity at the database level.

07. User Roles & Permissions

Role Hierarchy

```
Administrator  ->  Full platform control
      |
Seller         ->  Manages own listings + buyer interactions
      |
Buyer          ->  Browses, evaluates, and purchases vehicles
```

Permission Matrix

Action	Buyer	Seller	Admin
Register/Login	Yes	Yes	Yes
Browse vehicles	Yes	Yes	Yes
Create listing	No	Yes	Yes (override only)
Approve listing	No	No	Yes
Book inspection	Yes	No	No
Manage own bookings	Yes	View only	Yes
Wishlist vehicles	Yes	No	No
Submit review	Yes	No	No
View AI Price Prediction	Yes	Yes	Yes
View AI Fraud Risk	No	No	Yes
Verify sellers	No	No	Yes
Manage categories	No	No	Yes
Handle reports	No	No	Yes
Ban/suspend accounts	No	No	Yes
View platform analytics	No	Own listings only	Full access

Authentication & Authorization Approach

- **Authentication:** JWT (JSON Web Token) based login.
- **Authorization:** Each API endpoint checks the `role` claim in the JWT via FastAPI middleware/dependencies.
- **Ownership checks:** Endpoints additionally verify resource ownership (e.g., `seller_id == current_user.id`) before allowing edits.

```
Request -> JWT Validation -> Role Check -> Ownership Check -> Action Allowed/Denied
```

Why Role-Based Access Control Matters

- **Security:** Prevents cross-role access to private data or admin functions.
- **Data Integrity:** Only authorized users can approve listings or moderate content.
- **Trust:** Reinforces the platform's core goal of a safe, fraud-resistant marketplace.
- **Scalability:** Clean structure allows new roles to be added later without redesign.

08. Success Criteria

Functional Success Criteria

The project is functionally successful when users can:

- Register and log in securely using JWT authentication
- Create, edit, and delete vehicle listings (Seller)
- Upload multiple images per vehicle listing
- Browse and search vehicles using advanced filters
- Save favourite vehicles to a wishlist
- Book vehicle inspections and manage bookings
- Contact sellers through built-in messaging
- Submit reviews after a completed transaction
- Receive AI-generated fair price predictions
- View AI Vehicle Health Score
- View AI Fraud/Fake Listing Risk indicators
- View Seller Trust Score
- Receive personalized vehicle recommendations
- Discover nearby vehicles via Google Maps integration
- Receive real-time notifications
- Allow administrators to approve/reject listings, verify sellers, and manage reports

AI Feature Success Criteria

AI Feature	Success Benchmark
Fair Price Prediction	Predicted range includes actual market price in >= 80% of test cases
Vehicle Health Score	Score correlates logically with condition, age, and mileage
Fake Listing Detection	Correctly flags >= 75% of seeded suspicious/fake test listings
Seller Trust Score	Updates dynamically based on reviews, verification, complaints
Recommendation Engine	Recommendations align with buyer preferences in user testing

Technical Success Criteria

- Backend APIs (FastAPI) respond correctly and handle errors gracefully
- MySQL schema supports all core entities without data integrity issues
- Frontend renders correctly across desktop and mobile
- RBAC functions correctly for Buyer, Seller, and Admin
- Image uploads via Cloudinary work reliably
- Google Maps integration displays nearby vehicles correctly
- Application is containerized with Docker and deployable
- No critical security vulnerabilities in the final build

Measurable Outcomes Summary

Category	Metric	Target
Core Features	% of listed features implemented	100% (v1.0 scope)
AI Price Prediction	Accuracy within predicted range	>= 80%
Fraud Detection	Detection rate on seeded test listings	>= 75%
Platform Stability	Critical bugs at submission/demo	0
Role Security	Unauthorized cross-role access succeeding	0
Responsiveness	UI usable on mobile and desktop	Yes
Deployment	Successfully deployed and accessible	Yes

09. Out of Scope

Excluded Features (v1.0)

- **Vehicle Loan Approval** — Bank/financial institution loan processing not included.
- **Insurance Processing** — Insurance purchase, renewal, or claims not supported.
- **Ownership Transfer via RTO** — Legal ownership transfer handled outside the platform.
- **Live GPS Vehicle Tracking** — Only static “nearby discovery” is supported.
- **Online Vehicle Auctions** — Fixed-price listings only, no bidding.
- **Cryptocurrency Payments** — Not supported in this version.
- **AI Image Generation for Advertisements** — Sellers must upload real photographs.
- **Autonomous Vehicle Integration** — Unrelated to the platform’s marketplace purpose.

Additional Exclusions

- **Full Payment Gateway Integration** — Payments table reserved in schema, but not implemented in v1.0.
- **Native Mobile Applications** — Responsive web app only for v1.0.
- **Multi-Language Support** — English only.
- **Multi-Seller Dealership Management Tools** — Individual seller accounts only.
- **Advanced Business Intelligence / Market Forecasting** — Beyond individual vehicle pricing.
- **In-App Voice/Video Calling** — Text-based messaging only.
- **Vehicle Exchange / Trade-In System** — Not supported in this version.

Future Scope Consideration

- Payment gateway integration (schema already reserved)
- Dealer/business account tier with bulk listing tools
- Native mobile applications
- Multi-language support
- Basic in-app calling/video inspection feature

10. Chosen Track

Selected Track

Full-Stack Web Application with Integrated AI/ML Services

AutoMartX follows a modern three-tier architecture: a React-based frontend, a Python/FastAPI backend, and a MySQL database, with AI/ML models integrated directly into the backend service layer.

Technology Stack

Backend

Technology	Purpose
Python	Core backend programming language
FastAPI	High-performance REST API framework
SQLAlchemy	ORM for database modeling and queries
PyMySQL	MySQL database driver for Python
Pydantic	Data validation and serialization
Uvicorn	ASGI server for running FastAPI

Frontend

Technology	Purpose
React.js	Component-based UI library
TypeScript	Type-safe JavaScript
Tailwind CSS	Utility-first CSS framework

Database

Technology	Purpose
MySQL	Primary relational database

Authentication & Security

Technology	Purpose
JWT	Stateless user authentication
Passlib / bcrypt	Password hashing

Cloud Storage & Location

Technology	Purpose
Cloudinary	Vehicle image and document storage/delivery
Google Maps API	Nearby vehicle discovery and location display

Artificial Intelligence / Machine Learning

Technology	Purpose
Scikit-learn	ML models for price prediction and health scoring
Pandas	Data processing and feature engineering
NumPy	Numerical computation

Deployment & Hosting

AutoMartX is deployed on **Railway**, using a public/private network architecture: only the frontend is exposed to the internet, while the backend and database run on Railway's private network. Edge-level CDN, WAF, and load balancing protect and optimize all public-facing traffic. An AWS-based stack is documented as the production-grade alternative.

Primary Hosting: Railway (Public/Private Split)

Component	Zone	Platform	Notes
CDN	Edge	Railway CDN	Caches static assets/HTML at edge; free on all plans
WAF	Edge	Railway WAF (Under Attack Mode)	DDoS/bot protection via browser challenge
Load Balancer	Edge	Railway Edge Network	Distributes traffic across replicas automatically
Frontend (React)	Public	Railway service (public domain)	Only publicly internet-facing component
Backend (FastAPI)	Private	Railway service (private networking)	No direct public internet access
MySQL Database	Private	Railway MySQL (private networking)	No direct public internet access
Image Storage	External	Cloudinary	Free tier, called from backend
Location Services	External	Google Maps API	Called from frontend/backend

Production Alternative: AWS Stack

Component	Zone	AWS Service
CDN	Edge	CloudFront
WAF	Edge	AWS WAF
Load Balancer	Edge	Application Load Balancer (ALB)
Frontend	Public	S3 + CloudFront
Backend	Private (subnet)	EC2 (Docker) or Elastic Beanstalk
Database	Private (subnet)	RDS for MySQL

Deployment Decision Summary

Criteria	Railway (Primary)	AWS (Alternative)
Setup time	Minutes	Hours

CDN/WAF/Load Balancing	Built-in, no configuration	Manual CloudFront/WAF/ALB setup
Public/Private isolation	Built-in private networking	Manual VPC/subnet configuration
Cost	Free tier available; Hobby plan $5/mo minimum	Free tier (12 months), then pay-as-you-go
Best for	Development, demo, capstone submission	Production launch, enterprise scalability

> ***Chosen approach:*** *AutoMartX is deployed on Railway, with the frontend publicly exposed behind Railway's CDN, WAF, and load balancer, and the backend and MySQL database isolated on private networking. The AWS stack is documented as the scalable production alternative.*

System Architecture Overview

```
User Browser
      |
      v
Railway Edge Network   <- CDN + WAF + Load Balancer
      |  (Public)
      v
React + TS + Tailwind  <- Frontend (PUBLIC)
      |  Private Network (REST API / JWT)
      v
FastAPI Backend        <- Backend (PRIVATE)
  |- Scikit-learn Models
      |  Private Network (SQLAlchemy / PyMySQL)
      v
MySQL Database          <- Database (PRIVATE)

External Services: Cloudinary (images) - Google Maps API (location)
```

Why This Track Was Chosen

- Full-stack breadth across frontend, backend, database, AI/ML, and cloud deployment.
- Python-centric AI integration avoids cross-language ML service calls.
- Industry-relevant stack: React, FastAPI, MySQL, Docker, and cloud hosting.
- Flexible deployment strategy documenting both free-tier and enterprise infrastructure.
- Scikit-learn models are appropriately scoped for the project's needs.

Summary

> *AutoMartX is built on a Python (FastAPI) + React (TypeScript) + MySQL full-stack foundation, enhanced with Scikit-learn-based AI services, and deployed on Railway with an AWS-based production alternative documented for future scaling.*

Appendix: System Architecture

User Workflow

Buyer Flow: Register/Login -> Browse & Search Vehicles -> View AI Price/Health/Trust Scores -> Save to Wishlist -> Book Inspection -> Contact Seller -> Complete Purchase -> Submit Review

Seller Flow: Register/Login -> Create Listing -> Upload Images & Documents -> Await Admin Approval -> Listing Goes Live -> Respond to Enquiries -> Manage Bookings -> Receive Reviews

Admin Flow: Login to Dashboard -> Review New Listings -> Verify Seller Documents -> Approve/Reject Listings -> Monitor Reports & Fraud Alerts -> Manage Platform

Database Architecture

The 12 core database tables and their relationships are detailed in full in Section 06 (Core Entities / Database Tables) above, including the complete MySQL `CREATE TABLE` schema.

Hosting & Deployment Architecture

The public/private network split, CDN, WAF, and load balancing configuration on Railway is detailed in full in Section 10 (Chosen Track) above.

This document consolidates the complete Problem Statement and Technical Documentation for the AutoMartX project.